AgroSmart – Análisis de Algoritmos y Estructuras de Datos

Módulo: Inventario (inventario.py)

1. Estructuras que usa el módulo

Estructura	Dónde	Para qué
Lista de diccionarios	`self.productos`	Guarda los productos (`nombre`, `cantidad`, `tipo`, `fecha_vencimiento`).
Pila	`PilaMovimientos` (`estructuras/pila_movimientos.py`)	Guarda los movimientos para poder deshacerlos.
Archivo JSON	`datos_inventario.json`	Guarda el inventario para que no se pierda al cerrar.

2. Registro de productos

Explicación. `registrar_producto` lee los campos, valida que nombre, cantidad y tipo no estén vacíos, que la cantidad sea un entero no negativo y que la fecha de vencimiento (si se escribió) tenga el formato AAAA-MM-DD. Luego agrega el producto a la lista, apila el movimiento y guarda el archivo.

Pseudocódigo

```
FUNCION registrar_producto
    LEER nombre, cantidad, tipo, fecha
    SI falta nombre, cantidad o tipo: MOSTRAR error y TERMINAR
    SI cantidad no es entero o es negativa: MOSTRAR error y TERMINAR
    SI fecha no está vacía y no tiene formato AAAA-MM-DD: MOSTRAR
error y TERMINAR
    producto = {nombre, cantidad, tipo, fecha}
    AGREGAR producto AL FINAL de productos
    APILAR {accion: "registrar", producto} en la pila
    GUARDAR archivo
    MOSTRAR inventario
FIN FUNCION
```

Complejidad. Tiempo O(1) para validar y agregar (guardar el archivo es O(n)). Espacio O(n).

Justificación. Una lista de diccionarios permite guardar varios productos con sus datos de forma sencilla, y agregar al final es rápido.

3. Mostrar inventario

Explicación. `mostrar_productos` recorre la lista con `enumerate()` y arma el texto que se ve en pantalla.

Pseudocódigo

```
FUNCION mostrar_productos
    SI productos está vacío: MOSTRAR "No hay productos" y TERMINAR
    PARA cada producto EN productos
        MOSTRAR número, nombre, cantidad, tipo y vencimiento
    FIN PARA
FIN FUNCION
```

Complejidad. Tiempo O(n). Espacio adicional O(n) por el texto generado.

Justificación. Se deben mostrar todos los productos, así que un recorrido lineal es lo mínimo necesario.

4. Eliminar el último producto

Explicación. Quita el último elemento con `pop()`, apila el movimiento (con el producto y la posición que tenía) y guarda el archivo.

Pseudocódigo

```
FUNCION eliminar_producto
    SI productos está vacío: MOSTRAR aviso y TERMINAR
    producto = SACAR el último de productos
    APILAR {accion: "eliminar", producto, posicion} en la pila
    GUARDAR archivo
FIN FUNCION
```

Complejidad. `pop()` es O(1); con el guardado del archivo, O(n).

5. Deshacer con una pila (LIFO)

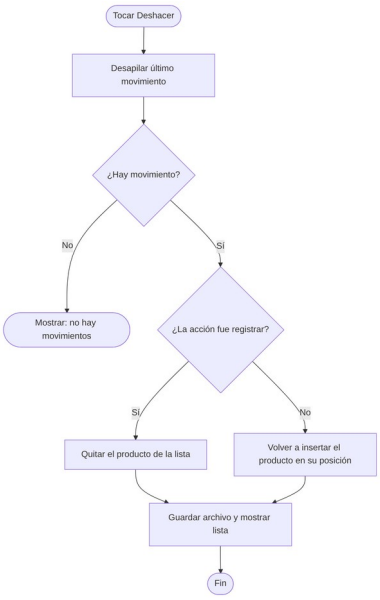
Explicación. Una **pila** es una estructura donde el último elemento que entra es el primero que sale (LIFO). Cada vez que se registra o elimina un producto, el movimiento se apila. Al tocar **Deshacer**, se desapila el último movimiento y se revierte: si fue un registro, se quita el producto; si fue una eliminación, se vuelve a insertar en su posición original.

Pseudocódigo

```
FUNCION deshacer_movimiento
    movimiento = pila.desapilar()
    SI movimiento es NADA: MOSTRAR "No hay movimientos" y TERMINAR
    SI movimiento.accion == "registrar"
        QUITAR movimiento.producto de productos
```

```
    SINO
        INSERTAR movimiento.producto en productos EN
movimiento.posicion
    FIN SI
    GUARDAR archivo
FIN FUNCION
```

Diagrama de flujo



Complejidad. Apilar y desapilar son O(1). Revertir un registro puede costar O(n) por buscar el producto en la lista; insertar en una posición es O(n). Espacio de la pila: O(m), con m = movimientos guardados.

Justificación. "Deshacer" siempre revierte lo más reciente, que es justo el comportamiento LIFO de una pila.

6. Guardado y carga (JSON)

Cada cambio escribe la lista completa en `datos_inventario.json`. Al iniciar, `cargar_datos` lee el archivo y valida cada registro. Si el archivo está dañado, el programa arranca con el inventario vacío en lugar de cerrarse. La pila de movimientos no se guarda: empieza vacía en cada sesión.